VORTEXTECH

Cyber Security Internship

Week 1 Report

Research on OWASP Top 10 Vulnerabilities

Presented By	Abdul Hadi Faheem
Track	Cyber Security Internship Track
Topic	Research and Documentation of Five OWASP Top 10 Vulnerabilities

Introduction:

Web applications are used for banking, shopping, education, healthcare, social media, and many other important activities. Because these applications store and process valuable information, security weaknesses can haveserious consequences. The **OWASP Top 10** is one of the most widely recognized awareness documents for common web application security risks.

This report discusses five important vulnerabilities from the OWASP Top 10:

1. Security Logging and Alerting Failures.
2. Software or Data Integrity Failures.
3. Authentication Failures.
4. Cryptographic Failures.
5. Injection.

Each vulnerability is explained in simple language, along with how an attacker could exploit it, a real-world example, and practical ways developers can prevent or reduce the risk.

1. Security Logging and Alerting Failures

What is it?

Security Logging and Alerting Failures occur when an organization does not properly record, monitor, or respond to important security events.

In simple words, it is like having security cameras in a building but nobody watching them. An attack may happen, but the organization may not notice it until serious damage has already occurred.

Applications should log important events such as:

- Successful and failed login attempts
- Password reset attempts
- Account or administrative changes
- Suspicious requests and activities
- System errors and important transactions

A security failure occurs when these events are not properly logged, monitored, or when alerts are ignored. As a result, attackers may remain undetected for a long time.

How Can an Attacker Exploit It?

Attackers can repeatedly attempt to access accounts using stolen usernames and passwords. If failed login attempts are not monitored, they may continue trying without being detected.

If an attacker successfully gains unauthorized access and suspicious activity does not generate alerts, they may remain inside the system for days or even months. During this time, they could steal sensitive data, install malware, or access other systems.

Poor incident response can also help attackers because suspicious activity may be recorded but not investigated quickly enough.

Real-World Example: Uber Data Security Incidents

A real world example involves security incidents at Uber. The U.S. Federal Trade Commission stated that Uber had weaknesses in monitoring access to sensitive consumer information and responding to certain security alerts.

These incidents demonstrate that simply collecting security logs is not enough. Organizations must actively monitor suspicious activity and respond quickly when a potential threat is detected.

Prevention and Mitigation

Developers and security teams can reduce this risk by:

- Logging successful and failed authentication attempts
- Monitoring logs for suspicious activity
- Creating alerts for unusual behavior
- Protecting logs from unauthorized modification
- Using centralized logging systems
- Securely storing important log backups
- Creating effective incident response procedures
- Regularly testing security alerts
- Avoiding sensitive information, such as passwords, in logs

Proper logging, monitoring, and alerting can significantly reduce the time an attacker remains undetected and help organizations respond before serious damage occurs.

2. Software or Data Integrity Failures

What is it?

Software or Data Integrity Failures occur when an application trusts software, updates, code, or data without properly verifying that they are authentic and have not been modified.
In simple words, imagine downloading a software update without checking whether it came from the original developer. If an attacker replaces it with malicious software, the victim may install malware while believing the update is legitimate.

This vulnerability can affect:

- Software updates and packages
- Third-party libraries and plugins
- CI/CD pipelines
- Downloaded files and code repositories
- Serialized or untrusted data

Modern applications depend on many external components. If developers do not verify their integrity and authenticity, attackers may compromise the application through the software supply chain.

How Can an Attacker Exploit It?

An attacker may compromise a software vendor, update server, or CI/CD pipeline and insert malicious code into legitimate software.

Users may then download and install the compromised update because they trust the original company. By compromising one trusted supplier, an attacker can potentially target many organizations at once.

Attackers can also manipulate untrusted or serialized data. If an application processes this data without proper validation, it may perform unintended actions or execute malicious code.

Real-World Example: SolarWinds Supply Chain Attack

The SolarWinds attack is a major example of a software integrity failure. Attackers compromised the SolarWinds Orion software supply chain and inserted malicious code into legitimate software updates.

Many customers downloaded the affected updates because they trusted SolarWinds. The attackers then used this access to target selected government agencies and private organizations.

This incident demonstrated that trusted software can become dangerous when attackers compromise the process used to build, sign, or distribute updates.

Prevention and Mitigation

Developers can reduce this risk by:

- Using trusted software repositories
- Verifying digital signatures and checksums
- Signing software releases and updates
- Protecting CI/CD and build systems
- Restricting access to build environments
- Reviewing third-party dependencies
- Regularly updating vulnerable components
- Validating untrusted data
- Using secure deserialization practices

Organizations should never automatically trust software, updates, or external components without verifying their authenticity and integrity.

3. Authentication Failures

What is it?

Authentication Failures occur when an application does not properly verify that a person is really the user they claim to be.

Authentication is the process of proving identity. For example, when a user enters a username and password, the application checks whether those credentials belong to a legitimate user.

Authentication failures can occur when an application:

- Allows weak passwords
- Uses default passwords
- Does not limit repeated login attempts
- Does not use Multi-Factor Authentication (MFA)
- Allows credential stuffing attacks
- Uses insecure password recovery
- Manages sessions insecurely
- Does not properly log users out

In simple words, authentication failures are like having a weak lock on the front door of a house. If the lock is easy to guess or bypass, an unauthorized person may enter.

How Can an Attacker Exploit It?

Attackers can perform a brute-force attack by repeatedly trying different passwords until they find the correct one.

They can also perform credential stuffing. In this attack, criminals use username and password combinations leaked from another website and try them on different services.

This is successful because many users reuse the same password across multiple websites.

An attacker may also exploit weak session management. For example, if a user logs out but the session token remains valid, another person using the same device might still access the account.

Real-World Example: Colonial Pipeline Attack

The Colonial Pipeline ransomware incident is a well known example showing the danger of compromised credentials and weak authentication protection.

Investigations and official statements indicated that attackers were able to use compromised credentials associated with a VPN account. The account could be accessed without Multi-Factor Authentication at the time.

This allowed attackers to gain access to the organization's network, eventually contributing to a major ransomware incident and operational disruption.

The case demonstrates why a password alone is often not enough to protect important systems.

Prevention and Mitigation

Developers and organizations should:

- Require strong and unique passwords
- Implement Multi-Factor Authentication
- Protect against brute-force attacks

- Limit or slow repeated failed login attempts
- Detect credential stuffing
- Avoid default credentials
- Secure password recovery processes
- Use secure session management
- Generate new session identifiers after authentication
- Expire sessions after inactivity
- Properly invalidate sessions after logout
- Monitor suspicious authentication activity

Using MFA is especially important because a stolen password alone may not be enough for an attacker to access an account.

4. Cryptographic Failures

What is it?

Cryptographic Failures occur when sensitive information is not properly protected using encryption and other security controls.

Sensitive data can include passwords, credit card numbers, personal information, financial records, and business secrets. Data must be protected both **at rest** (stored in files or databases) and **in transit** (moving across networks).

These failures can result from weak or outdated encryption, poor key management, weak password hashing, or failing to enforce HTTPS and TLS.

How Can an Attacker Exploit It?

An attacker on an insecure network, such as public Wi-Fi, may monitor traffic from websites that do not properly use HTTPS. This could expose usernames, passwords, session cookies, or personal information.

Attackers may also steal databases containing weakly protected passwords and attempt to crack them offline. Similarly, unencrypted files can expose sensitive information immediately if an attacker gains access.

Real-World Example: Uber Data Security Incidents

Uber's security incidents demonstrate the risks of insufficiently protected consumer data. Investigations described attackers gaining access to sensitive information, including names, email addresses, phone numbers, and driver license information.

This example shows that data exposure can become much more serious when sensitive information is not properly protected.

Prevention and Mitigation

Developers should:

- Encrypt sensitive data at rest
- Enforce HTTPS and modern TLS
- Use strong, trusted cryptographic algorithms
- Properly Protect Encryption Keys
- Use secure password hashing and salts
- Disable outdated encryption protocols
- Regularly review cryptographic configurations

Proper cryptography is essential because simply claiming that data is encrypted does not guarantee that it is secure.

5. Injection

What is it?

Injection occurs when an application accepts untrusted input and does not properly separate user data from commands.

A common example is **SQL Injection**, where an application directly combines user input with a database query. Specially crafted input may then be interpreted as a command instead of normal data.

Injection vulnerabilities can affect SQL databases, operating system commands, LDAP, NoSQL databases, and other interpreters.

How Can an Attacker Exploit It?

An attacker may provide specially crafted input that changes the logic of a database query or command.

If successful, the attacker could:

- Bypass authentication
- Access sensitive information
- Modify or delete data
- Access administrative functions

The main problem occurs when an application trusts user-controlled input and allows it to influence commands executed by another system.

Real-World Example: Heartland Payment Systems Breach

The Heartland Payment Systems breach is a well-known example associated with SQL injection. Attackers exploited a vulnerability to gain unauthorized access to the organization's environment, eventually contributing to the theft of payment card information.

The incident affected a large number of payment cards and demonstrated how a vulnerability in a web application can lead to serious financial and data security consequences.

Prevention and Mitigation

The most important protection methods are:

- Parameterized queries
- Prepared statements

Other security practices include:

- Validating user input
- Using secure APIs
- Avoiding unnecessary dynamic queries
- Applying least privilege access to databases
- Performing regular security testing
- Using Web Application Firewalls as an additional security layer

Developers should always treat user input as untrusted.

Personal Reflection

The vulnerability that surprised me the most was **Software or Data Integrity Failures**. I was surprised to learn that even trusted software can become dangerous if attackers compromise its update or supply chain. The SolarWinds attack showed how one compromised update can affect many organizations. This research helped me understand the importance of verifying the security and integrity of third-party software.

Conclusion

The OWASP Top 10 helps developers understand common web application security risks. The five vulnerabilities discussed in this report show that attacks can result from weak logging, compromised software, poor authentication, weak cryptography, and unsafe user input.

Developers can reduce these risks by following secure coding practices, using modern security controls, regularly testing applications, and treating security as an essential part of software development.